



JAVASCRIPT MASTER STUDY DOCUMENTATION

CHAPTER 08: STRINGS & TEXT PROCESSING

TOTAL SECTIONS

75 Modules + Projects

CURRICULUM PART

Part 04 — Core Data Structures
(1/3)

CORE MECHANICS

Immutability, Unicode & Text
Parsing

REFERENCE STANDARD

ECMAScript 2026 / V8 Runtime

Strings — String / টেক্সট নিয়ে কাজ

JavaScript-এ **String** হলো text বা character-এর sequence। যেমন:

JAVASCRIPT

EXAMPLES

```
let name = "Shariar";  
let university = "Daffodil International University";  
let message = "Hello JavaScript!";
```

এগুলো সবই **String**।

PART 01 — STRING CREATION, QUOTES & ESCAPE CHARACTERS (8.1 – 8.9)

8.1 String কী?

সহজভাবে:

Core Definition: String = এক বা একাধিক character-এর collection যা text represent করে।

REAL-LIFE EXAMPLE

একটি website-এ:

```
Name: Shariar Ahamed Ripon  
Email: ripon@example.com  
Address: Dhaka, Bangladesh
```

এখানে `Name`, `Email`, `Address` -এর value—সবই String।

8.2 String তৈরি করার ৩টি উপায়

JavaScript-এ String সাধারণত তিনভাবে লেখা যায়:

1. Double Quotes

JAVASCRIPT

1. DOUBLE QUOTES

```
let name = "Shariar";
```

2. Single Quotes

JAVASCRIPT

2. SINGLE QUOTES

```
let name = 'Shariar';
```

3. Backticks

JAVASCRIPT

3. BACKTICKS

```
let name = `Shariar`;
```

তিনটিই valid।

8.3 Single Quote vs Double Quote

JAVASCRIPT

CODE

```
let name1 = "Shariar";
let name2 = 'Shariar';

console.log(name1);
console.log(name2);
```

OUTPUT

Shariar Shariar

কাজের দিক থেকে দুটোই প্রায় একই।

8.4 Backticks

Backtick:

```
`
```

এটি বিশেষভাবে গুরুত্বপূর্ণ কারণ এর মাধ্যমে **Template Literal** ব্যবহার করা যায়।

JAVASCRIPT

CODE

```
let name = `Shariar`;

console.log(name);
```

Output:

Shariar

Template Literal আমরা পরে বিস্তারিত দেখব।

8.5 String-এর মধ্যে Quote ব্যবহার

ধরো text-এর মধ্যে quotation mark লাগবে।

JAVASCRIPT

CODE

```
let message = 'He said "Hello"';

console.log(message);
```

OUTPUT

He said "Hello"

অথবা:

JAVASCRIPT

CODE

```
let message = "It's a beautiful day";

console.log(message);
```

Output:

It's a beautiful day

8.6 Escape Character

String-এর মধ্যে একই ধরনের quote ব্যবহার করতে হলে \ ব্যবহার করা যায়।

JAVASCRIPT

CODE

```
let message = "He said \"Hello\"";  
  
console.log(message);
```

OUTPUT

He said "Hello"

এখানে:

\"

মানে quote-টিকে String-এর শেষ হিসেবে না ধরে character হিসেবে ব্যবহার করো।

8.7 Common Escape Characters

Escape	অর্থ
\"	Double quote
\'	Single quote
\\	Backslash
\n	New line
\t	Tab
\r	Carriage return
\b	Backspace
\f	Form feed

8.8 New Line — \n

JAVASCRIPT

CODE

```
let message = "Hello\nJavaScript";  
  
console.log(message);
```

OUTPUT

Hello JavaScript

Real-life example:

JAVASCRIPT

CODE

```
let address = "Dhaka\nBangladesh";  
  
console.log(address);
```

Output:

Dhaka Bangladesh

8.9 Tab — \t

JAVASCRIPT

CODE

```
console.log("Name:\tShariar");
```

Output:

```
Name:   Shariar
```

PART 02 — LENGTH, INDEXING & CHARACTER ACCESS (8.10 – 8.20)

8.10 String Length

String-এর character সংখ্যা জানতে:

JAVASCRIPT

CODE

```
.length
```

ব্যবহার করা হয়।

JAVASCRIPT

CODE

```
let name = "Shariar";  
  
console.log(name.length);
```

OUTPUT

```
7
```

কারণ:

```
S h a r i a r 1 2 3 4 5 6 7
```

8.11 Space-ও Character

JAVASCRIPT

CODE

```
let name = "Shariar Ahamed";  
  
console.log(name.length);
```

Space-ও count হবে।

8.12 String Index

JavaScript-এ String-এর প্রতিটি character-এর একটি index থাকে।

Index শুরু হয়:

```
0
```

থেকে।

JAVASCRIPT

CODE

```
let name = "Ripon";
```

এটা:

```
R i p o n  
0 1 2 3 4
```

8.13 Character Access — []

JAVASCRIPT

CODE

```
let name = "Ripon";

console.log(name[0]);
console.log(name[1]);
console.log(name[4]);
```

OUTPUT

```
R i n
```

8.14 প্রথম Character

JAVASCRIPT

CODE

```
let name = "JavaScript";

console.log(name[0]);
```

Output:

```
J
```

8.15 শেষ Character

শেষ character বের করতে:

JAVASCRIPT

CODE

```
let name = "JavaScript";

console.log(name[name.length - 1]);
```

Output:

```
t
```

কারণ length:

```
10
```

শেষ index:

```
10 - 1 = 9
```

8.16 .at() Method

শেষ character আরও সহজে:

JAVASCRIPT

CODE

```
let name = "JavaScript";

console.log(name.at(-1));
```

Output:

```
t
```

`at()` negative index support করে।

JAVASCRIPT

CODE

```
console.log(name.at(-2));
```

Output:

```
p
```

8.17 charAt()

JAVASCRIPT

CODE

```
let name = "Ripon";

console.log(name.charAt(0));
console.log(name.charAt(2));
```

Output:

```
R
P
```

[] বনাম charAt()

JAVASCRIPT

[] বনাম CHARAT()

```
let name = "Ripon";

console.log(name[10]);
console.log(name.charAt(10));
```

Output:

```
undefined
```

charAt() invalid index-এ empty string ("") দেয়, আর [] সাধারণত undefined দেয়।

8.18 String Concatenation

দুই বা তার বেশি String একসাথে যুক্ত করাকে **Concatenation** বলে।

JAVASCRIPT

CODE

```
let firstName = "Shariar";
let lastName = "Ripon";

let fullName = firstName + " " + lastName;

console.log(fullName);
```

OUTPUT

```
Shariar Ripon
```

8.19 String + Number

JAVASCRIPT

CODE

```
let age = 23;

console.log("Age: " + age);
```

Output:

```
Age: 23
```

JavaScript এখানে number-কে String-এর সাথে concatenate করেছে।

8.20 += দিয়ে String যোগ

JAVASCRIPT

CODE

```
let message = "Hello";

message += " ";
message += "JavaScript";

console.log(message);
```

Output:

```
Hello JavaScript
```

PART 03 — TEMPLATE LITERALS & CASE TRANSFORMATION (8.21 – 8.31)

8.21 Template Literals

Modern JavaScript-এ String তৈরি করার সবচেয়ে useful featureগুলোর একটি হলো:

Core Definition: Template Literal

Backtick ব্যবহার করতে হয়:

JAVASCRIPT

CODE

```
` `
```

EXAMPLE

JAVASCRIPT

EXAMPLE

```
let name = "Shariar";
let age = 23;

let message = `My name is ${name} and I am ${age} years old.`;

console.log(message);
```

OUTPUT

```
My name is Shariar and I am 23 years old.
```

8.22 \${} কী?

Template Literal-এর মধ্যে:

JAVASCRIPT

CODE

```
${variable}
```

ব্যবহার করে variable-এর value বসানো যায়।

JAVASCRIPT

CODE

```
let product = "Laptop";
let price = 75000;

console.log(`Product: ${product}`);
console.log(`Price: ${price} BDT`);
```

Output:

```
Product: Laptop
Price: 75000 BDT
```

8.23 Expression-এর ব্যবহার

শুধু variable নয়, expression-ও দেওয়া যায়।

JAVASCRIPT

CODE

```
let a = 10;
let b = 20;

console.log(`Total = ${a + b}`);
```

Output:

```
Total = 30
```

8.24 Function-এর Result

JAVASCRIPT

CODE

```
function getName() {
  return "Shariar";
}

console.log(`Hello ${getName()}`);
```

Output:

```
Hello Shariar
```

8.25 Multiline String

Normal quote-এ:

JAVASCRIPT

CODE

```
let message = "Hello
JavaScript";
```

এভাবে সরাসরি লিখলে error হবে।

কিন্তু Template Literal:

JAVASCRIPT

CODE

```
let message = `Hello
JavaScript
World`;

console.log(message);
```

Output:

```
Hello
JavaScript
World
```

8.26 toUpperCase()

String-এর সব letter uppercase করতে:

JAVASCRIPT

CODE

```
let name = "shariar";

console.log(name.toUpperCase());
```

Output:

```
SHARIAR
```

8.27 toLowerCase()

JAVASCRIPT

CODE

```
let name = "SHARIAR";  
  
console.log(name.toLowerCase());
```

Output:

```
shariar
```

8.28 Real-life Example — Login

User email:

JAVASCRIPT

CODE

```
let email = "USER@GMAIL.COM";  
  
console.log(email.toLowerCase());
```

Output:

```
user@gmail.com
```

Login system-এ case normalization-এর মতো কাজে এটি useful হতে পারে।

8.29 trim()

User input-এর শুরু বা শেষে extra space থাকলে:

JAVASCRIPT

CODE

```
let username = "  Shariar  ";  
  
console.log(username.trim());
```

Output:

```
Shariar
```

এটি:

```
"  Shariar  "
```

কে:

```
"Shariar"
```

করে।

8.30 trimStart()

শুধু শুরু থেকে whitespace remove:

JAVASCRIPT

CODE

```
let text = "  Hello";  
  
console.log(text.trimStart());
```

Output:

```
Hello
```

8.31 trimEnd()

শুধু শেষ থেকে:

JAVASCRIPT

CODE

```
let text = "Hello  ";  
  
console.log(text.trimEnd());
```

Output:

Hello

PART 04 — SEARCHING, CHECKING & SUBSTRING EXTRACTION (8.32 – 8.45)

8.32 includes()

কোনো String-এর মধ্যে নির্দিষ্ট text আছে কিনা check করতে:

JAVASCRIPT

CODE

```
let message = "I love JavaScript";  
  
console.log(message.includes("JavaScript"));
```

Output:

true

না থাকলে:

JAVASCRIPT

CODE

```
console.log(message.includes("Python"));
```

Output:

false

8.33 Real-life Example — Search

JAVASCRIPT

CODE

```
let title = "JavaScript Complete Course";  
  
let search = "JavaScript";  
  
if (title.includes(search)) {  
  console.log("Match found");  
}
```

Output:

Match found

8.34 startsWith()

String কোনো নির্দিষ্ট text দিয়ে শুরু হয়েছে কিনা:

JAVASCRIPT

CODE

```
let url = "https://example.com";  
  
console.log(url.startsWith("https"));
```

Output:

true

8.35 endsWith()

JAVASCRIPT

CODE

```
let file = "profile.jpg";  
  
console.log(file.endsWith(".jpg"));
```

Output:

```
true
```

Real-life:

JAVASCRIPT

CODE

```
let file = "document.pdf";  
  
if (file.endsWith(".pdf")) {  
  console.log("PDF file");  
}
```

8.36 indexOf()

কোনো text কোথায় আছে তা বের করতে:

JAVASCRIPT

CODE

```
let text = "I love JavaScript";  
  
console.log(text.indexOf("JavaScript"));
```

Output:

```
7
```

কারণ "JavaScript" শুরু হয়েছে index 7 থেকে।

8.37 indexOf() না পেলো

JAVASCRIPT

CODE

```
let text = "I love JavaScript";  
  
console.log(text.indexOf("Python"));
```

Output:

```
-1
```

অর্থাৎ:

```
-1 = পাওয়া যায়নি
```

8.38 lastIndexOf()

একই text একাধিকবার থাকলে শেষ occurrence-এর index পাওয়া যায়।

JAVASCRIPT

CODE

```
let text = "JavaScript is great. JavaScript is powerful.";  
  
console.log(text.lastIndexOf("JavaScript"));
```

Output:

```
21
```

8.39 search()

JAVASCRIPT

CODE

```
let text = "I love JavaScript";  
  
console.log(text.search("JavaScript"));
```

Output:

7

search() পরে Regex-এর সাথে আরও useful হবে।

8.40 slice()

String-এর একটি অংশ বের করতে:

JAVASCRIPT

CODE

```
slice(start, end)
```

EXAMPLE

JAVASCRIPT

EXAMPLE

```
let text = "JavaScript";  
  
console.log(text.slice(0, 4));
```

Output:

Java

মনে রাখবে:

Core Definition: end index include হয় না।

8.41 slice() Example

JAVASCRIPT

CODE

```
let text = "JavaScript";  
  
console.log(text.slice(4, 10));
```

Output:

Script

Index:

```
J a v a S c r i p t  
0 1 2 3 4 5 6 7 8 9
```

8.42 Negative slice()

JAVASCRIPT

CODE

```
let text = "JavaScript";  
  
console.log(text.slice(-6));
```

Output:

Script

শেষ ৬টি character।

8.43 substring()

JAVASCRIPT

CODE

```
let text = "JavaScript";  
  
console.log(text.substring(0, 4));
```

Output:

Java

substring() negative index-কে 0 হিসেবে treat করে।

8.44 slice() vs substring()

Feature	slice()	substring()
Negative index	✓	✗
End excluded	✓	✓
সাধারণ ব্যবহার	বেশি	কম

Modern code-এ অনেক ক্ষেত্রে slice() বেশি convenient।

8.45 substr() — পুরোনো Method

JAVASCRIPT

CODE

```
text.substr(start, length)
```

এটি পুরোনো/deprecated API।

নতুন code-এ সাধারণত ব্যবহার না করে:

JAVASCRIPT

CODE

```
slice()
```

ব্যবহার করা ভালো।

PART 05 — REPLACING, SPLITTING, PADDING & IMMUTABILITY (8.46 – 8.58)

8.46 replace()

String-এর একটি অংশ replace করতে:

JAVASCRIPT

CODE

```
let text = "I love JavaScript";  
  
let result = text.replace("JavaScript", "Python");  
  
console.log(result);
```

Output:

I love Python

8.47 Important — `replace()` Original String পরিবর্তন করে না

JAVASCRIPT

CODE

```
let text = "Hello World";

let result = text.replace("World", "JavaScript");

console.log(text);
console.log(result);
```

Output:

```
Hello World
Hello JavaScript
```

কারণ String **immutable**।

8.48 String Immutability

এটি খুব গুরুত্বপূর্ণ।

String তৈরি হওয়ার পর তার character সরাসরি পরিবর্তন করা যায় না।

JAVASCRIPT

CODE

```
let name = "Ripon";

name[0] = "S";

console.log(name);
```

Output:

```
Ripon
```

R → S হয়নি।

8.49 তাহলে String পরিবর্তন কীভাবে হয়?

নতুন String তৈরি হয়।

JAVASCRIPT

CODE

```
let name = "Ripon";

name = "Sharlar";

console.log(name);
```

এখানে পুরোনো String পরিবর্তন না হয়ে নতুন value assign হয়েছে।

8.50 `replaceAll()`

সব occurrence replace করতে:

JAVASCRIPT

CODE

```
let text = "apple apple apple";

console.log(text.replaceAll("apple", "banana"));
```

Output:

```
banana banana banana
```

8.51 concat()

দুটি String যুক্ত করতে:

```
JAVASCRIPT CODE
let first = "Hello";
let second = "World";

console.log(first.concat(" ", second));
```

Output:

```
Hello World
```

তবে modern JavaScript-এ সাধারণত:

```
JAVASCRIPT CODE
first + " " + second
```

অথবা template literal বেশি readable।

8.52 split()

String-কে Array-তে ভাগ করতে:

```
JAVASCRIPT CODE
let text = "HTML CSS JavaScript";

let result = text.split(" ");

console.log(result);
```

Output:

```
["HTML", "CSS", "JavaScript"]
```

8.53 split() with Comma

```
JAVASCRIPT CODE
let skills = "HTML,CSS,JavaScript,React";

console.log(skills.split(","));
```

Output:

```
["HTML", "CSS", "JavaScript", "React"]
```

এটি real-world data processing-এ খুব useful।

8.54 String → Array → String

```
JAVASCRIPT CODE
let skills = "HTML,CSS,JavaScript";

let arr = skills.split(",");

console.log(arr);

let result = arr.join(" | ");

console.log(result);
```

Output:

```
["HTML", "CSS", "JavaScript"]
```

```
HTML | CSS | JavaScript
```

8.55 repeat()

একটি String নির্দিষ্ট সংখ্যকবার repeat করতে:

JAVASCRIPT

CODE

```
let text = "Hi ";  
  
console.log(text.repeat(3));
```

Output:

```
Hi Hi Hi
```

8.56 padStart()

String-এর শুরুতে character যোগ করে নির্দিষ্ট length করা যায়।

JAVASCRIPT

CODE

```
let number = "5";  
  
console.log(number.padStart(3, "0"));
```

Output:

```
005
```

8.57 padEnd()

JAVASCRIPT

CODE

```
let number = "5";  
  
console.log(number.padEnd(3, "0"));
```

Output:

```
500
```

8.58 Real-life Example — Invoice Number

JAVASCRIPT

CODE

```
let invoiceNumber = "25";  
  
let formatted = invoiceNumber.padStart(5, "0");  
  
console.log(formatted);
```

Output:

```
00025
```

8.59 `charCodeAt()`

Character-এর UTF-16 code unit value পাওয়া যায়।

```
JAVASCRIPT CODE  
  
let letter = "A";  
  
console.log(letter.charCodeAt(0));
```

Output:

```
65
```

আর:

```
JAVASCRIPT CODE  
  
console.log("a".charCodeAt(0));
```

Output:

```
97
```

8.60 `String.fromCharCode()`

Code value থেকে String character তৈরি করা যায়।

```
JAVASCRIPT CODE  
  
console.log(String.fromCharCode(65));
```

Output:

```
A
```

8.61 **Unicode**

JavaScript String Unicode text handle করতে পারে।

```
JAVASCRIPT CODE  
  
let text = "বাংলাদেশ";  
  
console.log(text);
```

Output:

```
বাংলাদেশ
```

আর:

```
JAVASCRIPT CODE  
  
let text = "Hello 🍌";  
  
console.log(text);
```

Output:

```
Hello 🍌
```

8.62 localeCompare()

দুটি String compare করতে:

JAVASCRIPT

CODE

```
console.log("apple".localeCompare("banana"));
```

সাধারণত negative result পাওয়া যাবে, কারণ "apple" আগে।

JAVASCRIPT

CODE

```
console.log("banana".localeCompare("apple"));
```

সাধারণত positive result।

JAVASCRIPT

CODE

```
console.log("apple".localeCompare("apple"));
```

Output:

```
0
```

Sorting-এর সময় এটি useful।

8.63 Case-sensitive Comparison

JAVASCRIPT

CODE

```
console.log("hello" === "Hello");
```

Output:

```
false
```

কারণ:

```
h ≠ H
```

8.64 Case-insensitive Comparison

দুটিকে একই case-এ convert করে compare করতে পারো:

JAVASCRIPT

CODE

```
let a = "Hello";  
let b = "HELLO";  
  
console.log(a.toLowerCase() === b.toLowerCase());
```

Output:

```
true
```

8.65 String Object बनाम Primitive String

साधारणतः:

JAVASCRIPT

CODE

```
let name = "Ripon";
```

एक primitive String।

अथवा:

JAVASCRIPT

CODE

```
let name = new String("Ripon");
```

एक String object।

Important

साधारण application code-ए:

JAVASCRIPT

IMPORTANT

```
new String()
```

व्यवहार न करें।

Primitive String व्यवहार करें:

JAVASCRIPT

IMPORTANT

```
let name = "Ripon";
```

8.66 typeof String

JAVASCRIPT

CODE

```
let name = "Ripon";  
  
console.log(typeof name);
```

Output:

```
string
```

किन्तु:

JAVASCRIPT

CODE

```
let name = new String("Ripon");  
  
console.log(typeof name);
```

Output:

```
object
```

8.67 String Conversion

Number → String:

JAVASCRIPT

CODE

```
let age = 23;

console.log(String(age));
```

Output:

```
"23"
```

অথবা:

JAVASCRIPT

CODE

```
console.log(age.toString());
```

Output:

```
"23"
```

8.68 Real-life Example — User Input

Browser-এর form input সাধারণত String হিসেবে আসে।

JAVASCRIPT

CODE

```
let age = "23";

console.log(typeof age);
```

Output:

```
string
```

Number হিসেবে দরকার হলে:

JAVASCRIPT

CODE

```
let numericAge = Number(age);

console.log(numericAge);
console.log(typeof numericAge);
```

Output:

```
23
number
```

8.69 String Methods একসাথে

একটি String:

JAVASCRIPT

CODE

```
let text = "  JavaScript is Awesome  ";
```

এর উপর:

JAVASCRIPT

CODE

```
console.log(text.trim());
console.log(text.toUpperCase());
console.log(text.toLowerCase());
console.log(text.includes("JavaScript"));
console.log(text.startsWith("  Java"));
console.log(text.endsWith("  "));
```

এভাবে বিভিন্ন operation করা যায়।

8.70 Real-life Project Example — Username Formatter

ধরো user লিখেছে:

```
SHARIAR AHAMED RIPON
```

আমরা চাই:

```
shariar ahamed ripon
```

Code:

JAVASCRIPT

CODE

```
let username = "  SHARIAR AHAMED RIPON  ";

let formattedUsername = username
  .trim()
  .toLowerCase();

console.log(formattedUsername);
```

Output:

```
shariar ahamed ripon
```

8.71 Real-life Example — Email Normalization

JAVASCRIPT

CODE

```
let email = "  USER@GMAIL.COM  ";

email = email.trim().toLowerCase();

console.log(email);
```

Output:

```
user@gmail.com
```

8.72 Real-life Example — File Extension

JAVASCRIPT

CODE

```
let fileName = "profile.png";

if (fileName.endsWith(".png")) {
  console.log("PNG image detected");
}
```

Output:

```
PNG image detected
```

8.73 Real-life Example — Search Box

JAVASCRIPT

CODE

```
let product = "HP Laptop Core i5";

let search = "laptop";

if (product.toLowerCase().includes(search.toLowerCase())) {
  console.log("Product found");
}
```

Output:

```
Product found
```

ଏହି e-commerce search functionality-এর basic concept।

8.74 Real-life Example — Generate Profile Message

JAVASCRIPT

CODE

```
let name = "Shariar";
let skill = "Full Stack Development";
let experience = 1;

let message = `
Hello, I am ${name}.
I am learning ${skill}.
Experience: ${experience} year.
`;

console.log(message);
```

Output:

```
Hello, I am Shariar.
I am learning Full Stack Development.
Experience: 1 year.
```

8.75 String Methods Cheat Sheet 🗨

Method	কাজ
<code>.length</code>	String-এর length
<code>[index]</code>	Character access
<code>.at()</code>	Character access
<code>.charAt()</code>	Character access
<code>.toUpperCase()</code>	Uppercase
<code>.toLowerCase()</code>	Lowercase
<code>.trim()</code>	দুই পাশের whitespace remove
<code>.trimStart()</code>	শুরু থেকে whitespace remove
<code>.trimEnd()</code>	শেষ থেকে whitespace remove
<code>.includes()</code>	text আছে কিনা
<code>.startsWith()</code>	শুরু check
<code>.endsWith()</code>	শেষ check
<code>.indexOf()</code>	প্রথম occurrence-এর index
<code>.lastIndexOf()</code>	শেষ occurrence-এর index
<code>.search()</code>	Search
<code>.slice()</code>	অংশ বের করা
<code>.substring()</code>	অংশ বের করা
<code>.replace()</code>	একটি occurrence replace
<code>.replaceAll()</code>	সব occurrence replace
<code>.concat()</code>	String যোগ
<code>.split()</code>	String → Array
<code>.repeat()</code>	Repeat
<code>.padStart()</code>	শুরুতে padding
<code>.padEnd()</code>	শেষে padding
<code>.charCodeAt()</code>	Character code
<code>String.fromCharCode()</code>	Code → Character
<code>.localeCompare()</code>	String comparison

MENTAL MAP 🔥 **String-এর সবচেয়ে গুরুত্বপূর্ণ বিষয় (Architecture Tree)**

এই flow-টা মনে রাখো:

```
String
├── Create
│   ├── "Hello"
│   ├── 'Hello'
│   └── `Hello`
├── Access
│   ├── [index]
│   ├── at()
│   └── charAt()
├── Information
│   └── length
├── Transform
│   ├── toUpperCase()
│   ├── toLowerCase()
│   ├── trim()
│   └── replace()
├── Search
│   ├── includes()
│   ├── indexOf()
│   ├── startsWith()
│   └── endsWith()
├── Extract
│   ├── slice()
│   └── substring()
├── Convert
│   └── split()
└── Template
    └── `${}`
```

String Immutable

JAVASCRIPT

STRING IMMUTABLE

```
let text = "Hello";

text[0] = "Y";

console.log(text);
```

Output:

Hello

String-এর character সরাসরি পরিবর্তন করা যায় না।

slice() end index include করে না

JAVASCRIPT

SLICE() END INDEX INCLUDE করে না

```
let text = "JavaScript";

console.log(text.slice(0, 4));
```

Output:

Java

includes() Boolean দেয়

JAVASCRIPT

INCLUDES() BOOLEAN দেয়

```
"JavaScript".includes("Script");
```

Result:

true

indexOf() না পেলে -1

JAVASCRIPT

INDEXOF() না পেলে -1

```
"Hello".indexOf("x");
```

Result:

-1

split() String-কে Array বানায়

JAVASCRIPT

SPLIT() STRING-কে ARRAY বানায়

```
"HTML,CSS,JS".split(",");
```

Result:

JAVASCRIPT

SPLIT() STRING-কে ARRAY বানায়

```
["HTML", "CSS", "JS"]
```

Practice 1

Output কী হবে?

JAVASCRIPT

CODE

```
let name = "JavaScript";

console.log(name.length);
console.log(name[0]);
console.log(name[name.length - 1]);
```

Practice 2

Output কী হবে?

JAVASCRIPT

CODE

```
let text = "Hello JavaScript";

console.log(text.toUpperCase());
console.log(text.toLowerCase());
```

Practice 3

User input:

JAVASCRIPT

CODE

```
let username = " SHARIAR  ";
```

এমন code লেখো যাতে output হয়:

Practice 4

Check করো:

JAVASCRIPT

CODE

```
let email = "user@gmail.com";
```

Email-এর মধ্যে @ আছে কিনা।

Practice 5

Check করো:

JAVASCRIPT

CODE

```
let file = "profile.jpg";
```

এটি .jpg file কিনা।

Practice 6

এই String থেকে:

JAVASCRIPT

CODE

```
let text = "JavaScript";
```

শুধু:

বের করো।

Practice 7

JAVASCRIPT

CODE

```
let skills = "HTML,CSS,JavaScript,React";
```

এটিকে Array বানাও।

Expected:

JAVASCRIPT

CODE

```
["HTML", "CSS", "JavaScript", "React"]
```

Practice 8

নিচের output তৈরি করো:

```
My name is Shariar.  
I am 23 years old.  
I am learning JavaScript.
```

Template Literal ব্যবহার করে।

Practice 9 — Mini Project

একটি product data:

JAVASCRIPT

CODE

```
let productName = " HP LAPTOP ";  
let price = 75000;  
let category = "electronics";
```

Output:

```
Product: HP LAPTOP  
Price: 75000 BDT  
Category: ELECTRONICS
```

এখানে ব্যবহার করবে:

- `trim()`
- `toUpperCase()`
- Template Literal

SUMMARY 🚀 Chapter 8 Final Summary

এই chapter-এ আমরা শিখলাম:

- ✓ String কী
- ✓ Single / Double / Backtick
- ✓ Escape Characters
- ✓ String Length
- ✓ Indexing
- ✓ at()
- ✓ charAt()
- ✓ Concatenation
- ✓ Template Literals
- ✓ \${expression}
- ✓ Multiline Strings
- ✓ toUpperCase()
- ✓ toLowerCase()
- ✓ trim()
- ✓ includes()
- ✓ startsWith()
- ✓ endsWith()
- ✓ indexOf()
- ✓ lastIndexOf()
- ✓ search()
- ✓ slice()
- ✓ substring()
- ✓ replace()
- ✓ replaceAll()
- ✓ concat()
- ✓ split()
- ✓ repeat()
- ✓ padStart()
- ✓ padEnd()
- ✓ charCodeAt()
- ✓ Unicode
- ✓ localeCompare()
- ✓ String Conversion
- ✓ String Immutability

★ সবচেয়ে বেশি ব্যবহার হবে

JAVASCRIPT

★ সবচেয়ে বেশি ব্যবহার হবে

```
length
[]
at()
toUpperCase()
toLowerCase()
trim()
includes()
startsWith()
endsWith()
indexOf()
slice()
replace()
split()
```

NEXT CHAPTER 🚀 Chapter 9 Preview — Arrays

পরবর্তী অধ্যায় হবে **Chapter 9 — Arrays**, যেখানে আমরা Array তৈরি, indexing, `push()`, `pop()`, `shift()`, `unshift()`, `slice()`, `splice()`, `find()`, `filter()`, `map()`, `reduce()`, `sort()`, `forEach()`, nested arrays, array destructuring এবং real-life projects সহ বিস্তারিত শিখব।